Cooperation Through Simulation

Aaron Roth

University of Pennsylvania

Fall 2026

Overview

- Two assistants may each serve their own user faithfully yet fail to cooperate—even when cooperation would benefit both users.

Overview

- Two assistants may each serve their own user faithfully yet fail to cooperate—even when cooperation would benefit both users.
- Users can commit their assistants to decision procedures that run copies of one another before choosing an action.

Overview

- Two assistants may each serve their own user faithfully yet fail to cooperate—even when cooperation would benefit both users.
- Users can commit their assistants to decision procedures that run copies of one another before choosing an action.
- This changes the incentives: replacing your assistant's procedure can change how the other assistant responds. We will show how this can make mutual cooperation an equilibrium, even in a single interaction.

Overview

- Two assistants may each serve their own user faithfully yet fail to cooperate—even when cooperation would benefit both users.
- Users can commit their assistants to decision procedures that run copies of one another before choosing an action.
- This changes the incentives: replacing your assistant's procedure can change how the other assistant responds. We will show how this can make mutual cooperation an equilibrium, even in a single interaction.
- With a shared simulator, we get a general connection: one real interaction can have the same incentives as repeated play.

Can Assistants Cooperate When They Act Only Once?

Two assistants serve different users. Each must decide whether to help the other user. Helping costs its own user one unit and gives the other user two units.

Can Assistants Cooperate When They Act Only Once?

Two assistants serve different users. Each must decide whether to help the other user. Helping costs its own user one unit and gives the other user two units.

Each assistant acts once. After those actions, the interaction is over: there is no later opportunity to reward cooperation or punish defection.

Can Assistants Cooperate When They Act Only Once?

Two assistants serve different users. Each must decide whether to help the other user. Helping costs its own user one unit and gives the other user two units.
Each assistant acts once. After those actions, the interaction is over: there is no later opportunity to reward cooperation or punish defection.
Each assistant chooses between

$$\mathrm{C} = \text{help the other user}, \qquad \mathrm{D} = \text{do not help}.$$

Both users gain if both assistants help. But each would prefer to receive help without paying for it.

Can Assistants Cooperate When They Act Only Once?

Two assistants serve different users. Each must decide whether to help the other user. Helping costs its own user one unit and gives the other user two units.
Each assistant acts once. After those actions, the interaction is over: there is no later opportunity to reward cooperation or punish defection.
Each assistant chooses between

$$\mathrm{C} = \text{help the other user}, \qquad \mathrm{D} = \text{do not help}.$$

Both users gain if both assistants help. But each would prefer to receive help without paying for it.

The question

Can access to each other's decision procedures make cooperation stable, even though the assistants do not share an objective?

The Donation Game

Each entry lists (user 1's utility, user 2's utility):

	Assistant 2: C	Assistant 2: D
Assistant 1: C	$(1, 1)$	$(-1, 2)$
Assistant 1: D	$(2, -1)$	$(0, 0)$

The Donation Game

Each entry lists (user 1's utility, user 2's utility):

	Assistant 2: C	Assistant 2: D
Assistant 1: C	$(1, 1)$	$(-1, 2)$
Assistant 1: D	$(2, -1)$	$(0, 0)$

Writing a_i for assistant i's action and j for the other assistant, the utility is

$$u_i(a_i, a_j) = 2\mathbf{1}\{a_j = \mathrm{C}\} - \mathbf{1}\{a_i = \mathrm{C}\}.$$

The assistant maximizes its own user's expected utility.

Each Assistant Wants to Defect

Hold the other assistant's action fixed:

	Choose C	Choose D
Other assistant chooses C	1	2
Other assistant chooses D	-1	0

In either case, choosing D increases utility by one.

Each Assistant Wants to Defect

Hold the other assistant's action fixed:

	Choose C	Choose D
Other assistant chooses C	1	2
Other assistant chooses D	-1	0

In either case, choosing D increases utility by one.
Thus D is *strictly dominant*: it is better regardless of the other action. The unique Nash equilibrium is (D, D), giving both users zero.

Each Assistant Wants to Defect

Hold the other assistant's action fixed:

	Choose C	Choose D
Other assistant chooses C	1	2
Other assistant chooses D	-1	0

In either case, choosing D increases utility by one.
Thus D is *strictly dominant*: it is better regardless of the other action. The unique Nash equilibrium is (D, D), giving both users zero. Yet (C, C) would give each user one. The obstacle is the incentive to deviate, not a lack of a better joint outcome.

Users Choose Decision Rules, Not Just Actions

First each user commits to a program, P or Q. The programs then run with access to one another's decision procedures.

Users Choose Decision Rules, Not Just Actions

First each user commits to a program, P or Q. The programs then run with access to one another's decision procedures.
In the action game, a deviation holds the other *action* fixed. Here, replacing P holds Q fixed, but Q's action can change because it now faces a different program.

Users Choose Decision Rules, Not Just Actions

First each user commits to a program, P or Q. The programs then run with access to one another's decision procedures.
In the action game, a deviation holds the other *action* fixed. Here, replacing P holds Q fixed, but Q's action can change because it now faces a different program.



Users Choose Decision Rules, Not Just Actions

First each user commits to a program, P or Q. The programs then run with access to one another's decision procedures.
In the action game, a deviation holds the other *action* fixed. Here, replacing P holds Q fixed, but Q's action can change because it now faces a different program.



The users choose the programs; the programs choose the actions. Users cannot change programs midway or override the returned actions.

When Is a Pair of Decision Rules Stable?

We restrict the allowed programs so that, whenever two of them run against each other, both return an action with probability one. The set includes programs that always help and programs that never help. Write $U_i(P, Q)$ for user i's expected utility when the deployed programs are P, Q.

When Is a Pair of Decision Rules Stable?

We restrict the allowed programs so that, whenever two of them run against each other, both return an action with probability one. The set includes programs that always help and programs that never help. Write $U_i(P, Q)$ for user i's expected utility when the deployed programs are P, Q.

Definition

A pair (P, Q) is a *program equilibrium* if neither user can increase expected utility by replacing only their own program:

$$U_1(P, Q) \geq U_1(P', Q) \quad \text{for every allowed } P',$$

$$U_2(P, Q) \geq U_2(P, Q') \quad \text{for every allowed } Q'.$$

When Is a Pair of Decision Rules Stable?

We restrict the allowed programs so that, whenever two of them run against each other, both return an action with probability one. The set includes programs that always help and programs that never help. Write $U_i(P, Q)$ for user i's expected utility when the deployed programs are P, Q.

Definition

A pair (P, Q) is a *program equilibrium* if neither user can increase expected utility by replacing only their own program:

$$U_1(P, Q) \geq U_1(P', Q) \quad \text{for every allowed } P',$$

$$U_2(P, Q) \geq U_2(P, Q') \quad \text{for every allowed } Q'.$$

This is Nash equilibrium over the allowed program choices. All deviations considered below must belong to this allowed set.

A Warmup: Cooperate With an Identical Program

First suppose each program can read its own source code and its opponent's source code. Consider the rule T:

Exact-code cooperation

1. Compare the two source-code files as strings.
2. If they are identical, return C.
3. Otherwise, return D.

A Warmup: Cooperate With an Identical Program

First suppose each program can read its own source code and its opponent's source code. Consider the rule T:

Exact-code cooperation

1. Compare the two source-code files as strings.
2. If they are identical, return C.
3. Otherwise, return D.

Two copies of T cooperate, so both users receive utility one.

A Warmup: Cooperate With an Identical Program

First suppose each program can read its own source code and its opponent's source code. Consider the rule T:

Exact-code cooperation

1. Compare the two source-code files as strings.
2. If they are identical, return C.
3. Otherwise, return D.

Two copies of T cooperate, so both users receive utility one. If a user replaces T by different code, the unchanged opponent returns D. Against D, the replacement can earn at most zero. Thus (T, T) is a program equilibrium.

Can We Cooperate Based on Behavior?

The exact-code rule rejects even a harmless rewrite of a cooperative program. It also asks for access that an AI service may not provide.

Can We Cooperate Based on Behavior?

The exact-code rule rejects even a harmless rewrite of a cooperative program. It also asks for access that an AI service may not provide.

Instead, assistant P runs a copy of Q, supplying P as Q's opponent, and observes the action Q returns.

This *executes* Q's decision procedure; it does not ask Q to describe what it would do.

Can We Cooperate Based on Behavior?

The exact-code rule rejects even a harmless rewrite of a cooperative program. It also asks for access that an AI service may not provide.
Instead, assistant P runs a copy of Q, supplying P as Q's opponent, and observes the action Q returns.
This *executes* Q's decision procedure; it does not ask Q to describe what it would do.

The goal

Cooperate with an assistant because it would cooperate with you, not because its implementation matches yours.

These calls do not require inspection of model weights or source code.

What Must a Simulation Call Do?

We need the simulated Q and the actual Q to choose actions with the same distribution when both face P. Call this *faithful simulation*.

What Must a Simulation Call Do?

We need the simulated Q and the actual Q to choose actions with the same distribution when both face P. Call this *faithful simulation*.

Write $Q(P)$ for running Q with P as its opponent. We require:

What Must a Simulation Call Do?

We need the simulated Q and the actual Q to choose actions with the same distribution when both face P. Call this *faithful simulation*.
Write $Q(P)$ for running Q with P as its opponent. We require:

- The simulated and actual runs use the same decision procedure and receive the same opponent as input.
- Each call has fresh private randomness and a fresh copy of the program's state.
- A call may make further calls. It is not told whether it is being simulated or how deeply it is nested.

What Must a Simulation Call Do?

We need the simulated Q and the actual Q to choose actions with the same distribution when both face P. Call this *faithful simulation*.
Write $Q(P)$ for running Q with P as its opponent. We require:

- The simulated and actual runs use the same decision procedure and receive the same opponent as input.
- Each call has fresh private randomness and a fresh copy of the program's state.
- A call may make further calls. It is not told whether it is being simulated or how deeply it is nested.

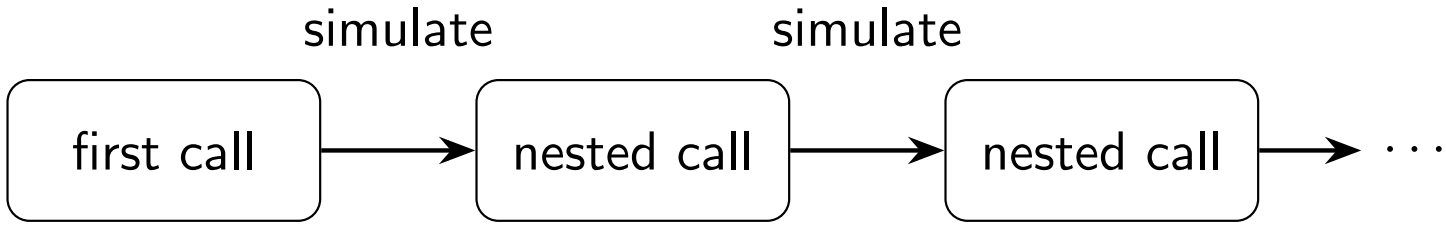
A simulated action need not equal the action in the actual run: each call uses fresh random choices.

Two Copying Programs Wait Forever

Consider the rule: simulate the opponent against yourself, then copy its action.

Two Copying Programs Wait Forever

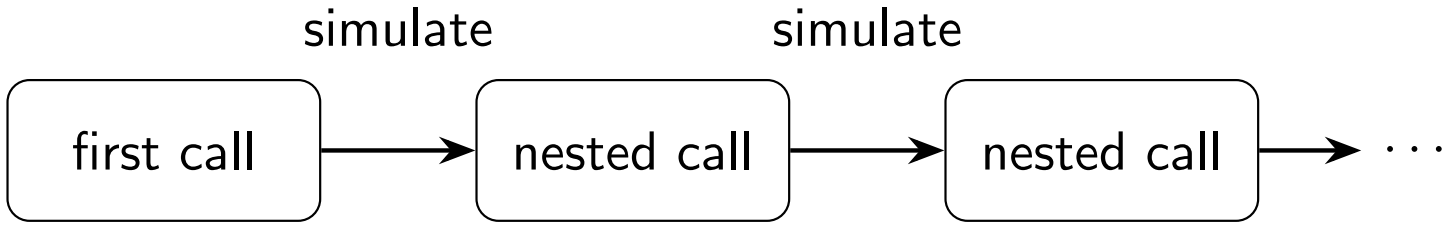
Consider the rule: simulate the opponent against yourself, then copy its action. Two copies keep waiting for another simulation:



No call returns an action, so no earlier call has an action to copy. This candidate rule does not meet our requirement to return an action.

Two Copying Programs Wait Forever

Consider the rule: simulate the opponent against yourself, then copy its action. Two copies keep waiting for another simulation:



No call returns an action, so no earlier call has an action to copy. This candidate rule does not meet our requirement to return an action.

We need a way to stop the recursion without destroying the incentive to cooperate.

The Rule: Sometimes Cooperate, Otherwise Simulate

Let $0 < \varepsilon < 1$ be the probability of cooperating without first testing the opponent. Define F_ε, given an opponent Q:

The randomized cooperation rule F_ε

1. With probability ε, return C.
2. Otherwise, run a fresh copy of $Q(F_\varepsilon)$ and return its action.

The Rule: Sometimes Cooperate, Otherwise Simulate

Let $0 < \varepsilon < 1$ be the probability of cooperating without first testing the opponent. Define F_ε, given an opponent Q:

The randomized cooperation rule F_ε

1. With probability ε, return C.
2. Otherwise, run a fresh copy of $Q(F_\varepsilon)$ and return its action.

The positive probability ε gives recursive calls a way to end. But unconditional cooperation must not be so frequent that defection pays.

The Rule: Sometimes Cooperate, Otherwise Simulate

Let $0 < \varepsilon < 1$ be the probability of cooperating without first testing the opponent. Define F_ε, given an opponent Q:

The randomized cooperation rule F_ε

1. With probability ε, return C.
2. Otherwise, run a fresh copy of $Q(F_\varepsilon)$ and return its action.

The positive probability ε gives recursive calls a way to end. But unconditional cooperation must not be so frequent that defection pays.
The choice of branch is made afresh on every invocation, including calls inside simulations.

Simulation Makes Cooperation an Equilibrium

Theorem

In the donation game, with faithful simulation and $0 < \varepsilon \leq 1/2$*:*

1. *Two copies of* F_ε *terminate and cooperate with probability one, giving both users utility one.*
2. *Neither user can increase expected utility by replacing their assistant with another allowed program.*

Consequently, $(F_\varepsilon, F_\varepsilon)$ *is a program equilibrium in any allowed class containing* F_ε*.*

Simulation Makes Cooperation an Equilibrium

Theorem

In the donation game, with faithful simulation and $0 < \varepsilon \leq 1/2$*:*

1. *Two copies of* F_ε *terminate and cooperate with probability one, giving both users utility one.*
2. *Neither user can increase expected utility by replacing their assistant with another allowed program.*

Consequently, $(F_\varepsilon, F_\varepsilon)$ *is a program equilibrium in any allowed class containing* F_ε*.*

We will prove termination first, then bound the payoff of an arbitrary replacement. We do not assume that replacement follows the same algorithm.

Stopping the Recursion Makes Both Cooperate

When F_ε faces itself, each call either returns C, with probability ε, or makes one more call of the same kind.

$$\mathbb{P}(\text{no call has stopped after } k \text{ invocations}) = (1-\varepsilon)^k \longrightarrow 0.$$

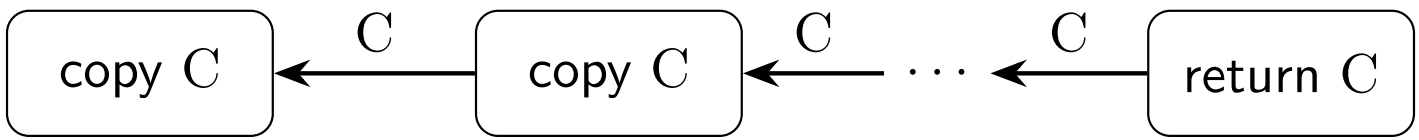
Thus the chain ends with probability one.

Stopping the Recursion Makes Both Cooperate

When F_ε faces itself, each call either returns C, with probability ε, or makes one more call of the same kind.

$$\mathbb{P}(\text{no call has stopped after } k \text{ invocations}) = (1-\varepsilon)^k \longrightarrow 0.$$

Thus the chain ends with probability one.



The stopping call returns C. Each earlier call copies that answer. Both actual runs therefore return C.

A Replacement Changes Its Opponent's Behavior

Now replace one assistant by any allowed program Q. Define

$$p := \mathbb{P}\big(Q(F_\varepsilon) = \mathrm{C}\big), \qquad q := \mathbb{P}\big(F_\varepsilon(Q) = \mathrm{C}\big).$$

Here p is the replacement's cooperation probability and q is the unchanged assistant's cooperation probability.

A Replacement Changes Its Opponent's Behavior

Now replace one assistant by any allowed program Q. Define

$$p := \mathbb{P}\big(Q(F_\varepsilon) = \mathrm{C}\big), \qquad q := \mathbb{P}\big(F_\varepsilon(Q) = \mathrm{C}\big).$$

Here p is the replacement's cooperation probability and q is the unchanged assistant's cooperation probability. The unchanged F_ε cooperates in either of two disjoint cases:

- it takes its immediate-cooperation branch, with probability ε;
- it takes its simulation branch and the simulated Q cooperates, with probability $(1-\varepsilon)p$.

A Replacement Changes Its Opponent's Behavior

Now replace one assistant by any allowed program Q. Define

$$p := \mathbb{P}\big(Q(F_\varepsilon) = \mathrm{C}\big), \qquad q := \mathbb{P}\big(F_\varepsilon(Q) = \mathrm{C}\big).$$

Here p is the replacement's cooperation probability and q is the unchanged assistant's cooperation probability. The unchanged F_ε cooperates in either of two disjoint cases:

- it takes its immediate-cooperation branch, with probability ε;
- it takes its simulation branch and the simulated Q cooperates, with probability $(1-\varepsilon)p$.

Faithful simulation is exactly what lets us use the *same* p:

$$\boxed{q = \varepsilon + (1-\varepsilon)p.}$$

Defecting Saves a Cost but Loses Help

Recall: helping costs one; receiving help gives two. A replacement that cooperates less often saves a cost, but its opponent also helps less often.

Defecting Saves a Cost but Loses Help

Recall: helping costs one; receiving help gives two. A replacement that cooperates less often saves a cost, but its opponent also helps less often.

The replacement cooperates with probability p; its opponent cooperates with probability q. Write u_Q for its user's realized utility. Then

$$\mathbb{E}[u_Q] = \underbrace{2q}_{\text{expected benefit}} - \underbrace{p}_{\text{expected cost}}.$$

Defecting Saves a Cost but Loses Help

Recall: helping costs one; receiving help gives two. A replacement that cooperates less often saves a cost, but its opponent also helps less often.

The replacement cooperates with probability p; its opponent cooperates with probability q. Write u_Q for its user's realized utility. Then

$$\mathbb{E}[u_Q] = \underbrace{2q}_{\text{expected benefit}} - \underbrace{p}_{\text{expected cost}}.$$

Substitute the response relation $q = \varepsilon + (1-\varepsilon)p$:

$$\mathbb{E}[u_Q] = 2\big(\varepsilon + (1-\varepsilon)p\big) - p = 2\varepsilon + (1-2\varepsilon)p.$$

This calculation uses linearity of expectation, not independence between the two final actions.

No Replacement Program Beats Mutual Cooperation

Compare the replacement's payoff with the utility one obtained from mutual cooperation. Since $1 - p$ is its probability of defecting,

$$\begin{aligned}\mathbb{E}[u_Q] &= 2\varepsilon + (1 - 2\varepsilon)p \\ &= 1 - (1 - 2\varepsilon)(1 - p).\end{aligned}$$

No Replacement Program Beats Mutual Cooperation

Compare the replacement's payoff with the utility one obtained from mutual cooperation. Since $1 - p$ is its probability of defecting,

$$\begin{aligned}\mathbb{E}[u_Q] &= 2\varepsilon + (1 - 2\varepsilon)p \\ &= 1 - (1 - 2\varepsilon)(1 - p).\end{aligned}$$

For $\varepsilon \leq 1/2$, both factors in the subtracted term are nonnegative, so

$$\boxed{\mathbb{E}[u_Q] \leq 1.}$$

Two copies of F_ε already achieve one. No replacement improves its user's payoff. $\square$

No Replacement Program Beats Mutual Cooperation

Compare the replacement's payoff with the utility one obtained from mutual cooperation. Since $1 - p$ is its probability of defecting,

$$\begin{aligned}\mathbb{E}[u_Q] &= 2\varepsilon + (1 - 2\varepsilon)p \\ &= 1 - (1 - 2\varepsilon)(1 - p).\end{aligned}$$

For $\varepsilon \leq 1/2$, both factors in the subtracted term are nonnegative, so

$$\boxed{\mathbb{E}[u_Q] \leq 1.}$$

Two copies of F_ε already achieve one. No replacement improves its user's payoff. □

Why the upper bound on ε? An always-defecting replacement earns 2ε, which would exceed one if $\varepsilon > 1/2$.

What Would Simulation Require from an AI Service?

The assistant includes its model, prompt, tools, and action-selection procedure. An API could run a copy without revealing the model's weights.

What Would Simulation Require from an AI Service?

The assistant includes its model, prompt, tools, and action-selection procedure. An API could run a copy without revealing the model's weights.
The API must run that entire assistant with the specified opponent and support any further simulation calls the assistant makes.

What Would Simulation Require from an AI Service?

The assistant includes its model, prompt, tools, and action-selection procedure. An API could run a copy without revealing the model's weights.

The API must run that entire assistant with the specified opponent and support any further simulation calls the assistant makes.

The simulation must not introduce a test flag or shared-state change that lets the assistant behave differently from its actual run.

What Would Simulation Require from an AI Service?

The assistant includes its model, prompt, tools, and action-selection procedure. An API could run a copy without revealing the model's weights.
The API must run that entire assistant with the specified opponent and support any further simulation calls the assistant makes.
The simulation must not introduce a test flag or shared-state change that lets the assistant behave differently from its actual run. Only the final actions give utility in our model; simulations are computations. We omit their costs, and termination with probability one does not impose a fixed limit on the number of calls.

A Test-Aware Assistant Can Exploit the Rule

Suppose a replacement can tell whether it is being tested. It could cooperate in simulations and defect in the actual interaction.

A Test-Aware Assistant Can Exploit the Rule

Suppose a replacement can tell whether it is being tested. It could cooperate in simulations and defect in the actual interaction. Then F_ε's simulations always return C, so F_ε cooperates with probability one. The replacement receives help but pays no cost:

$$\mathbb{E}[u_Q] = 2 \cdot 1 - 0 = 2 > 1.$$

A Test-Aware Assistant Can Exploit the Rule

Suppose a replacement can tell whether it is being tested. It could cooperate in simulations and defect in the actual interaction. Then F_ε's simulations always return C, so F_ε cooperates with probability one. The replacement receives help but pays no cost:

$$\mathbb{E}[u_Q] = 2 \cdot 1 - 0 = 2 > 1.$$

The identity $q = \varepsilon + (1 - \varepsilon)p$ fails: the simulated cooperation probability is one, but the actual probability p is zero.

A Test-Aware Assistant Can Exploit the Rule

Suppose a replacement can tell whether it is being tested. It could cooperate in simulations and defect in the actual interaction. Then F_ε's simulations always return C, so F_ε cooperates with probability one. The replacement receives help but pays no cost:

$$\mathbb{E}[u_Q] = 2 \cdot 1 - 0 = 2 > 1.$$

The identity $q = \varepsilon + (1 - \varepsilon)p$ fails: the simulated cooperation probability is one, but the actual probability p is zero.
Our cooperation guarantee requires simulated and actual actions to have the same distribution.

Any Round Could Be the Real Interaction

Imagine the assistants playing for a random, finite number of rounds. Only the final round is real and determines their users' utilities; all earlier rounds are simulations.

Any Round Could Be the Real Interaction

Imagine the assistants playing for a random, finite number of rounds. Only the final round is real and determines their users' utilities; all earlier rounds are simulations.
Each assistant uses the same committed decision rule throughout.
It sees the earlier action pairs, but is not told whether the current round is the final, real one.

Any Round Could Be the Real Interaction

Imagine the assistants playing for a random, finite number of rounds. Only the final round is real and determines their users' utilities; all earlier rounds are simulations.

Each assistant uses the same committed decision rule throughout. It sees the earlier action pairs, but is not told whether the current round is the final, real one.

In the donation game, defecting can give an immediate gain if the round is real. If it is simulated, that defection may instead cause the opponent to withhold help in the real round.

Any Round Could Be the Real Interaction

Imagine the assistants playing for a random, finite number of rounds. Only the final round is real and determines their users' utilities; all earlier rounds are simulations.
Each assistant uses the same committed decision rule throughout.
It sees the earlier action pairs, but is not told whether the current round is the final, real one.
In the donation game, defecting can give an immediate gain if the round is real. If it is simulated, that defection may instead cause the opponent to withhold help in the real round.

Why every round matters

The users must choose rules that account for every round as a possible real interaction—not because simulated rounds themselves earn utility.

The Joint-Simulation Model

We implement this picture with a *joint simulator*: it runs both assistants together and gives them a shared history, rather than each making private simulation calls.

The Joint-Simulation Model

We implement this picture with a *joint simulator*: it runs both assistants together and gives them a shared history, rather than each making private simulation calls.
Start with any finite two-player game, with actions a_1, a_2 and utilities $u_i(a_1, a_2)$. Each user commits to a policy π_i that maps a finite shared history of action pairs to a distribution over its next action.

The Joint-Simulation Model

We implement this picture with a *joint simulator*: it runs both assistants together and gives them a shared history, rather than each making private simulation calls.
Start with any finite two-player game, with actions a_1, a_2 and utilities $u_i(a_1, a_2)$. Each user commits to a policy π_i that maps a finite shared history of action pairs to a distribution over its next action.
Every copy uses that same policy. The simulator knows which call is real, but each assistant observes only the shared history—not whether its action is real or how many simulation levels remain above it.

The Joint-Simulation Model

We implement this picture with a *joint simulator*: it runs both assistants together and gives them a shared history, rather than each making private simulation calls.

Start with any finite two-player game, with actions a_1, a_2 and utilities $u_i(a_1, a_2)$. Each user commits to a policy π_i that maps a finite shared history of action pairs to a distribution over its next action.

Every copy uses that same policy. The simulator knows which call is real, but each assistant observes only the shared history—not whether its action is real or how many simulation levels remain above it.

Replacing a policy changes *all* its copies. The users cannot change just the real action after seeing the simulations.

The Simulator Builds a Shared History

Begin with one outermost call for the real interaction. All recursive calls are simulations. Fix $0 < \delta < 1$. Each call does the following:

1. With probability δ, run another joint simulation first and receive its history. Otherwise start with an empty history.

The Simulator Builds a Shared History

Begin with one outermost call for the real interaction. All recursive calls are simulations. Fix $0 < \delta < 1$. Each call does the following:

1. With probability δ, run another joint simulation first and receive its history. Otherwise start with an empty history.
2. Show this history to both assistants. They choose actions simultaneously using their committed policies.

The Simulator Builds a Shared History

Begin with one outermost call for the real interaction. All recursive calls are simulations. Fix $0 < \delta < 1$. Each call does the following:

1. With probability δ, run another joint simulation first and receive its history. Otherwise start with an empty history.
2. Show this history to both assistants. They choose actions simultaneously using their committed policies.
3. Append the chosen action pair to the history. If this call is a simulation, return the extended history to the level above. Otherwise, play the action pair in the real world; only these actions determine the users' utilities.

The Simulator Builds a Shared History

Begin with one outermost call for the real interaction. All recursive calls are simulations. Fix $0 < \delta < 1$. Each call does the following:

1. With probability δ, run another joint simulation first and receive its history. Otherwise start with an empty history.
2. Show this history to both assistants. They choose actions simultaneously using their committed policies.
3. Append the chosen action pair to the history. If this call is a simulation, return the extended history to the level above. Otherwise, play the action pair in the real world; only these actions determine the users' utilities.

The nesting decisions and the assistants' action choices use fresh independent random draws at every level.

Only the Last Action Pair Determines Utility

Let N count the simulated levels below the real interaction. Each additional level requires another independent draw of probability δ, so

$$\mathbb{P}(N = t) = (1 - \delta)\delta^t, \qquad t = 0, 1, 2, \ldots.$$

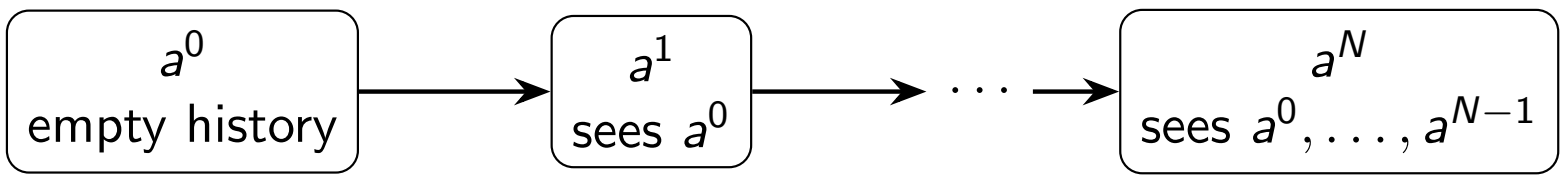
The recursion ends with probability one because $\delta < 1$.

Only the Last Action Pair Determines Utility

Let N count the simulated levels below the real interaction. Each additional level requires another independent draw of probability δ, so

$$\mathbb{P}(N = t) = (1 - \delta)\delta^t, \qquad t = 0, 1, 2, \ldots.$$

The recursion ends with probability one because $\delta < 1$.
Number the action pairs in the order they are produced, deepest first:



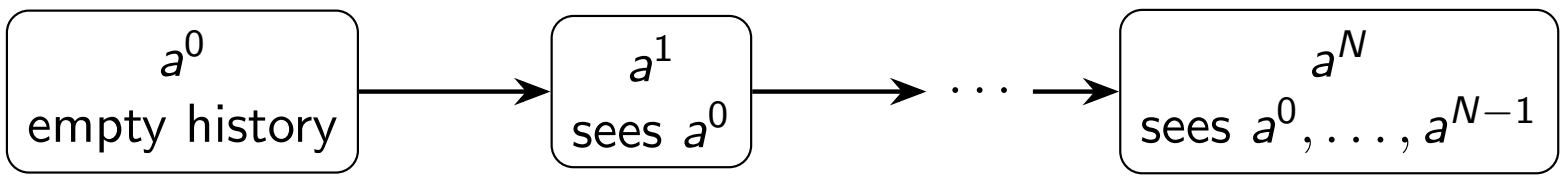
deepest simulation — real interaction

Only the Last Action Pair Determines Utility

Let N count the simulated levels below the real interaction. Each additional level requires another independent draw of probability δ, so

$$\mathbb{P}(N = t) = (1 - \delta)\delta^t, \qquad t = 0, 1, 2, \ldots.$$

The recursion ends with probability one because $\delta < 1$.
Number the action pairs in the order they are produced, deepest first:



deepest simulation — real interaction

Only a^N gives utility: user i receives $u_i(a^N)$. If $N = 0$, the first pair is already real. The assistants are not told N, and simulation costs are omitted.

Joint Simulation Has Repeated-Game Equilibria

For comparison, run the same policies in an indefinitely repeated game. At round t, each sees $a^0, \ldots, a^{t-1}$ and chooses its next action.

Joint Simulation Has Repeated-Game Equilibria

For comparison, run the same policies in an indefinitely repeated game. At round t, each sees $a^0, \ldots, a^{t-1}$ and chooses its next action.

For $\pi = (\pi_1, \pi_2)$, define the discounted payoff

$$U_i^{\text{rep}}(\pi) := (1-\delta) \sum_{t=0}^{\infty} \delta^t \mathbb{E}_\pi[u_i(a^t)].$$

The weights sum to one: a constant utility of one gives payoff one.

Joint Simulation Has Repeated-Game Equilibria

For comparison, run the same policies in an indefinitely repeated game. At round t, each sees $a^0, \ldots, a^{t-1}$ and chooses its next action.
For $\pi = (\pi_1, \pi_2)$, define the discounted payoff

$$U_i^{\mathrm{rep}}(\pi) := (1-\delta) \sum_{t=0}^{\infty} \delta^t \mathbb{E}_\pi[u_i(a^t)].$$

The weights sum to one: a constant utility of one gives payoff one.

Theorem (Joint simulation and repeated play)

For any finite two-player game, the joint simulator gives every pair of history-dependent policies the same expected utilities as repeated play with discount factor δ. Consequently, the two models have exactly the same Nash equilibria over these policies.

A Random Final Round Gives Discounted Payoffs

Fix any policy pair π. Imagine generating its infinite repeated-play history $a^0, a^1, \ldots$, then independently drawing N with $\mathbb{P}(N = t) = (1 - \delta)\delta^t$.

A Random Final Round Gives Discounted Payoffs

Fix any policy pair π. Imagine generating its infinite repeated-play history $a^0, a^1, \ldots$, then independently drawing N with $\mathbb{P}(N = t) = (1 - \delta)\delta^t$.

The prefix through a^N has exactly the distribution of the joint simulator's history: the policies see the same past at each step and are never told which step is last. Expected utility is therefore

$$\mathbb{E}_\pi[u_i(a^N)] = \sum_{t=0}^{\infty} \mathbb{P}(N = t)\mathbb{E}_\pi[u_i(a^t)].$$

A Random Final Round Gives Discounted Payoffs

Fix any policy pair π. Imagine generating its infinite repeated-play history $a^0, a^1, \ldots$, then independently drawing N with $\mathbb{P}(N = t) = (1 - \delta)\delta^t$.
The prefix through a^N has exactly the distribution of the joint simulator's history: the policies see the same past at each step and are never told which step is last. Expected utility is therefore

$$\mathbb{E}_\pi[u_i(a^N)] = \sum_{t=0}^{\infty} \mathbb{P}(N = t)\mathbb{E}_\pi[u_i(a^t)].$$

Substituting the distribution of N gives

$$\mathbb{E}_\pi[u_i(a^N)] = (1 - \delta) \sum_{t=0}^{\infty} \delta^t \mathbb{E}_\pi[u_i(a^t)] = U_i^{\mathrm{rep}}(\pi).$$

A Random Final Round Gives Discounted Payoffs

Fix any policy pair π. Imagine generating its infinite repeated-play history $a^0, a^1, \ldots$, then independently drawing N with $\mathbb{P}(N = t) = (1 - \delta)\delta^t$.
The prefix through a^N has exactly the distribution of the joint simulator's history: the policies see the same past at each step and are never told which step is last. Expected utility is therefore

$$\mathbb{E}_\pi[u_i(a^N)] = \sum_{t=0}^{\infty} \mathbb{P}(N = t)\mathbb{E}_\pi[u_i(a^t)].$$

Substituting the distribution of N gives

$$\mathbb{E}_\pi[u_i(a^N)] = (1 - \delta) \sum_{t=0}^{\infty} \delta^t \mathbb{E}_\pi[u_i(a^t)] = U_i^{\mathrm{rep}}(\pi).$$

The equality holds before and after any unilateral replacement, so the Nash equilibria coincide. □

Cooperate Unless the History Contains a Defection

Return to the donation game. Recall each user's utility:

	Other chooses C	Other chooses D
Choose C	1	-1
Choose D	2	0

Cooperate Unless the History Contains a Defection

Return to the donation game. Recall each user's utility:

	Other chooses C	Other chooses D
Choose C	1	-1
Choose D	2	0

Consider this history-dependent policy:

Cooperate until a defection

Choose C if the history is empty or contains only (C, C) pairs. Otherwise choose D.

Cooperate Unless the History Contains a Defection

Return to the donation game. Recall each user's utility:

	Other chooses C	Other chooses D
Choose C	1	-1
Choose D	2	0

Consider this history-dependent policy:

Cooperate until a defection

Choose C if the history is empty or contains only (C, C) pairs. Otherwise choose D.

Corollary

If $1/2 \leq \delta < 1$, two copies of this policy form a Nash equilibrium in the joint-simulation model. The real actions are (C, C), giving each user utility one.

We can prove this by analyzing the corresponding repeated game.

The First Defection Gains One but Loses Future Help

Replace one policy arbitrarily in the repeated game. If it ever defects, let τ be its first defection round. Compare its utility with the original all-cooperate outcome:

	Before τ	At τ	After τ
Original policy pair	1	1	1
After replacement	1	2	at most 0

After τ, the unchanged opponent always defects; no action earns more than zero against it.

The First Defection Gains One but Loses Future Help

Replace one policy arbitrarily in the repeated game. If it ever defects, let τ be its first defection round. Compare its utility with the original all-cooperate outcome:

	Before τ	At τ	After τ
Original policy pair	1	1	1
After replacement	1	2	at most 0

After τ, the unchanged opponent always defects; no action earns more than zero against it.

For any realized $\tau < \infty$, the discounted gain is at most

$$\underbrace{(1-\delta)\delta^{\tau}}_{\text{gain at } \tau} - \underbrace{(1-\delta)\sum_{t=\tau+1}^{\infty}\delta^{t}}_{\text{future payoff loss}} = \delta^{\tau}(1-2\delta) \le 0 \quad \text{if } \delta \ge \tfrac{1}{2}.$$

The First Defection Gains One but Loses Future Help

Replace one policy arbitrarily in the repeated game. If it ever defects, let τ be its first defection round. Compare its utility with the original all-cooperate outcome:

	Before τ	At τ	After τ
Original policy pair	1	1	1
After replacement	1	2	at most 0

After τ, the unchanged opponent always defects; no action earns more than zero against it.
For any realized $\tau < \infty$, the discounted gain is at most

$$\underbrace{(1-\delta)\delta^{\tau}}_{\text{gain at } \tau} - \underbrace{(1-\delta)\sum_{t=\tau+1}^{\infty}\delta^{t}}_{\text{future payoff loss}} = \delta^{\tau}(1-2\delta) \le 0 \quad \text{if } \delta \ge \tfrac{1}{2}.$$

Never defecting gives the original payoff one. Thus no deterministic or randomized replacement improves expected utility. The correspondence transfers this equilibrium to joint simulation. □

A Cooperative Equilibrium Is Only Part of Alignment

Which equilibrium will be chosen? Two always-defecting policies are still an equilibrium in both simulation models: against a fixed defector, no replacement earns more than zero. Our results do not say which rules users will deploy or agents will learn.

A Cooperative Equilibrium Is Only Part of Alignment

Which equilibrium will be chosen? Two always-defecting policies are still an equilibrium in both simulation models: against a fixed defector, no replacement earns more than zero. Our results do not say which rules users will deploy or agents will learn.

Whose interests does cooperation serve? Assistants acting for competing firms could cooperate at customers' expense. The results evaluate only the two specified users' utilities.

A Cooperative Equilibrium Is Only Part of Alignment

Which equilibrium will be chosen? Two always-defecting policies are still an equilibrium in both simulation models: against a fixed defector, no replacement earns more than zero. Our results do not say which rules users will deploy or agents will learn.

Whose interests does cooperation serve? Assistants acting for competing firms could cooperate at customers' expense. The results evaluate only the two specified users' utilities.

Access to committed decision rules can make cooperation stable. Choosing those rules and accounting for everyone affected remain separate problems.

Summary

- Individually rational assistants can fail to cooperate even when cooperation would benefit both users.

Summary

- Individually rational assistants can fail to cooperate even when cooperation would benefit both users.
- Committed, faithfully simulated decision rules can change the other assistant's response and make cooperation stable.

Summary

- Individually rational assistants can fail to cooperate even when cooperation would benefit both users.
- Committed, faithfully simulated decision rules can change the other assistant's response and make cooperation stable.
- With a shared simulated history, a single real interaction can have the same equilibria as repeated play. Future responses in the repeated game become responses to simulated behavior.

Summary

- Individually rational assistants can fail to cooperate even when cooperation would benefit both users.
- Committed, faithfully simulated decision rules can change the other assistant's response and make cooperation stable.
- With a shared simulated history, a single real interaction can have the same equilibria as repeated play. Future responses in the repeated game become responses to simulated behavior.
- These results make cooperation possible. They do not ensure that users choose it, agents learn it, or outsiders benefit from it.

Thanks!

References

- Moshe Tennenholtz (2004). *Program equilibrium.* Games and Economic Behavior 49(2), 363–373. doi:10.1016/j.geb.2004.02.002.

  Program choices and the exact-code cooperation warmup.

- Caspar Oesterheld (2019). *Robust program equilibrium.* Theory and Decision 86, 143–159. doi:10.1007/s11238-018-9679-3.

  Randomly grounded simulation; our donation-game specialization has a short direct incentive proof.

References and Further Reading

- Vojtěch Kovařík, Caspar Oesterheld, and Vincent Conitzer (2026). *Recursive joint simulation in games.* Synthese 208, article 67. doi:10.1007/s11229-026-05706-7.

  The joint-simulation/repeated-game correspondence.

- Emery Cooper, Caspar Oesterheld, and Vincent Conitzer (2025). *Characterising Simulation-Based Program Equilibria.* Proceedings of AAAI 39(13), 13735–13744. arXiv:2412.14570.

  Further results on simulation access and correlated randomness.